BEGINNER GUIDE

Haulix: how the website works

An HD planet behind glass, and an agent workstation that follows your scroll — explained in simple words with the code that matters.

Contents

01 What you are looking at
02 Why it looks good (design choices)
03 The HD Earth, step by step
04 How scrolling moves the planet
05 The workstation that follows your scroll
06 Small effects worth copying
07 How to change things
08 Words you'll meet

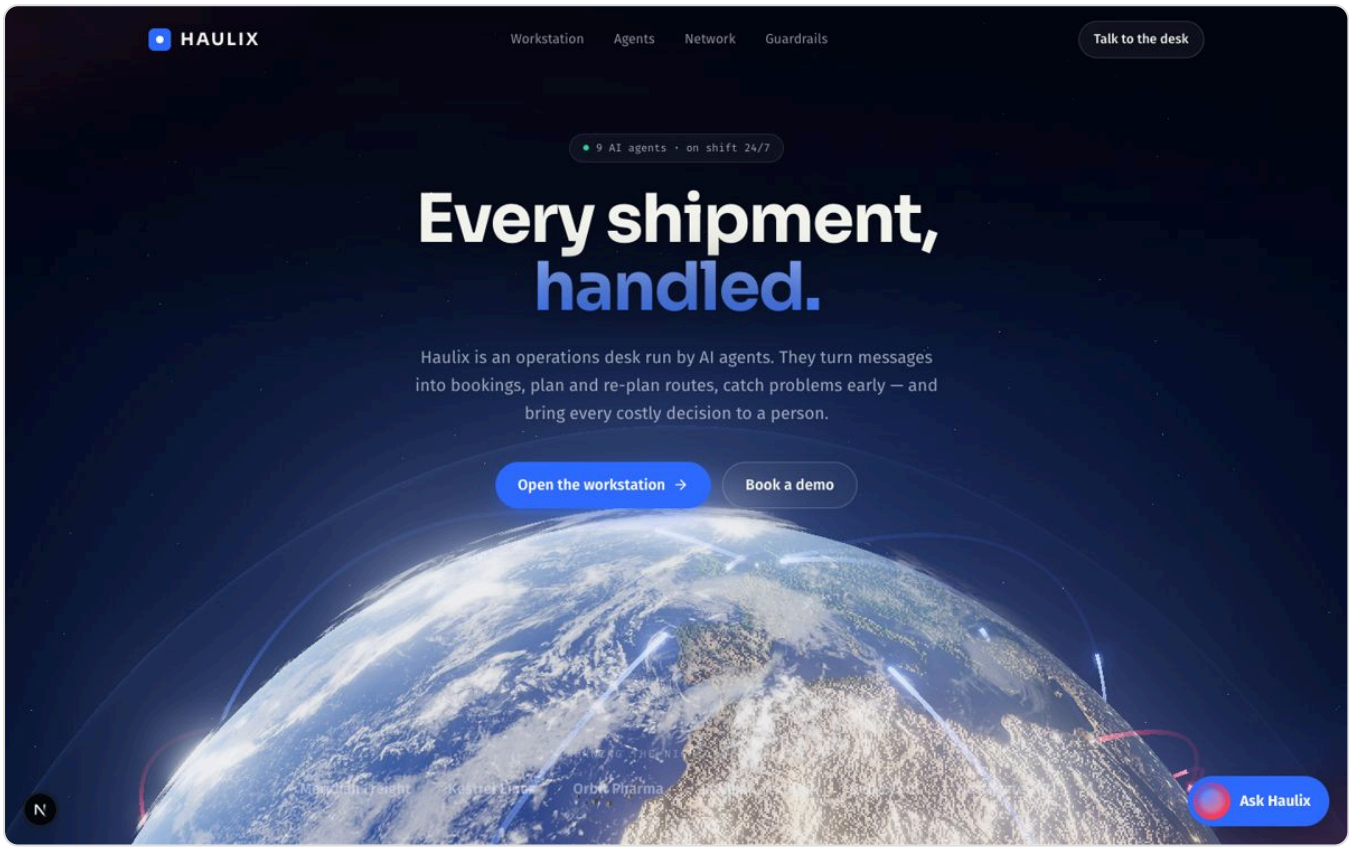
Written for Amartya Ghosh

1 Nov 2026 · haulix-ai-ops-desk.vercel.app

01 What you are looking at

Haulix is a website for an imaginary product: an **operations desk for logistics run by AI agents**. The agents read orders, plan truck routes and spot problems — and a person approves anything important.

The page has one big idea: **a glowing HD Earth floats behind everything**, and glass panels scroll over it. The middle of the page is the **agent workstation** — a fake but working app screen that changes as you scroll.



The hero: the Earth rises behind the headline. The network lines are real ports and airports.

Part of the page	What it shows	File
Earth (behind everything)	HD planet with clouds, night lights, shipping lanes	app/components/Planet.tsx
Hero	Headline, two buttons, customer names	app/components/Sections.tsx → Hero
Workstation	Live desk → Routes → Exceptions → Ops board, moved by scroll	Workstation.tsx + Sections.tsx → WorkstationSection
Agents	Nine glass cards, one per agent	Sections.tsx → Agents
Network	Animated bar chart, progress rows, counters	Sections.tsx → Network
Guardrails + CTA + footer	Rules for the agents, final call to action	Sections.tsx → Trust, Closing

Part of the page	What it shows	File
Text and demo data	Agent names, shipments, rules	app/data/site.ts

02 Why it looks good (design choices)

- **One hero object.** The Earth is the only 3D thing. Everything else is calm, flat glass. Too much 3D looks like a toy; one great object looks premium.
- **Dark deep-space background + one bright accent (electric blue).** Red is used only for problems (exceptions), green only for “OK”. Colour = meaning.
- **Glass panels** let the planet show through softly, so the page feels like one scene instead of separate boxes.
- **Big, tight headings** (font Sora, negative letter-spacing) and small mono labels (Fira Code) — a classic “tech product” pairing.
- **Content proves the product.** Instead of describing agents, the workstation *shows* them working — orders becoming bookings, routes re-planning.

> Rule of thumb you can reuse: pick ONE wow element, ONE accent colour, and let everything else be quiet.

03 The HD Earth, step by step

The planet is a sphere drawn with three.js. What makes it look real is **four images (textures)** from Solar System Scope (made from NASA data, free with credit):

Texture	Used for
day_8k.jpg (8192×4096)	The daytime colours of land and sea
night_8k.jpg	City lights on the dark side
ocean_4k.jpg	A mask: white = water. Used for sun glint and moving water
clouds_4k.jpg	Cloud shells and the shadows clouds cast on the ground

A **shader** (small graphics-card program) mixes them for every pixel. In simple words it asks: *is this spot facing the sun?* If yes, show the day colour; if no, darken it and add the city lights. Then it adds a blue rim at the edge (the atmosphere) and a sparkle where the sun reflects on water.

```
// Planet.tsx — inside the Earth shader (simplified)
float ndl = dot(normal, sunDirection);           // +1 facing the sun, −1 facing away
float dayAmt = smoothstep(−0.12, 0.32, ndl);     // soft line between day and night
vec3 col = dayColour ∗ (0.05 + 1.25 ∗ light);    // lit daytime colour
col = mix(col, col ∗ 0.03, nightAmount);         // darken the night side
col += cityLights ∗ 2.6 ∗ nightAmount;           // switch the cities on
col += blue ∗ rim;                               // glowing atmosphere at the edge
```

On top: two **cloud shells** (slightly bigger spheres that turn slowly), an **atmosphere glow**, a **starfield**, **pulsing hubs** at real ports, and **lanes** with a light running along them (red = a lane with a problem). Finally **bloom** makes bright things glow softly.

> The black-square glitch you saw was one broken pixel (a maths error) that bloom spread into a square. The fix: the shader now checks for broken numbers and replaces them before bloom sees them.

04 How scrolling moves the planet

The planet is fixed behind the page (`position: fixed`). Scrolling doesn't move the canvas — it changes the planet's **position and size** using a small list of keyframes:

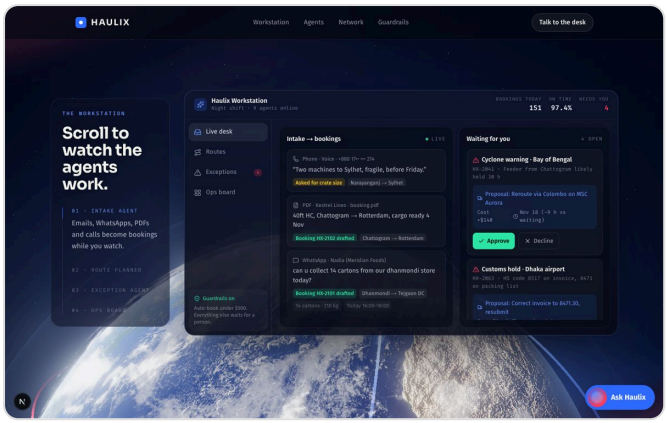
```
// Planet.tsx
const STOPS_X = [{p:0, v:0}, {p:0.3, v:−3.0}, {p:0.62, v:3.1}, {p:1, v:0}];   // left → right → centre
const STOPS_Y = [{p:0, v:−4.75}, {p:0.3, v:0.45}, {p:0.62, v:0.35}, {p:1, v:0.1}]; // starts low and huge
const STOPS_S = [{p:0, v:2.1}, {p:0.3, v:1.0}, {p:0.62, v:0.92}, {p:1, v:1.1}];  // scale

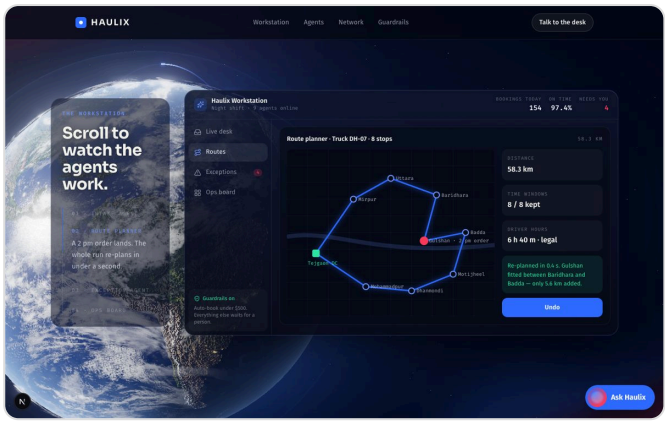
// every frame
p = window.scrollY / (page height − screen height);    // 0 at top, 1 at bottom
x = sample(STOPS_X, p);  y = sample(STOPS_Y, p);  s = sample(STOPS_S, p);
world.position.set(x, y + entry, 0);  world.scale.setScalar(s);
```

`sample()` blends smoothly between the two nearest keyframes. The values also **ease** toward the target each frame, so even a jumpy scroll looks smooth. When the page first loads, `entry` lifts the planet up from below over 1.9 seconds — the “rising Earth” moment.

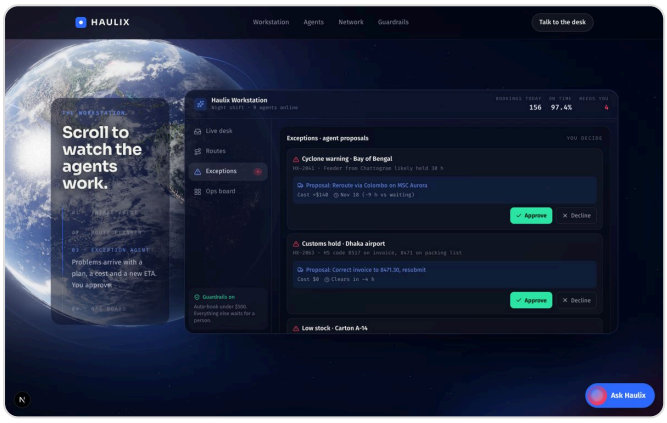
> Try it: change `v: −3.0` to `v: −1.5` in STOPS_X and the planet swings less far to the left.

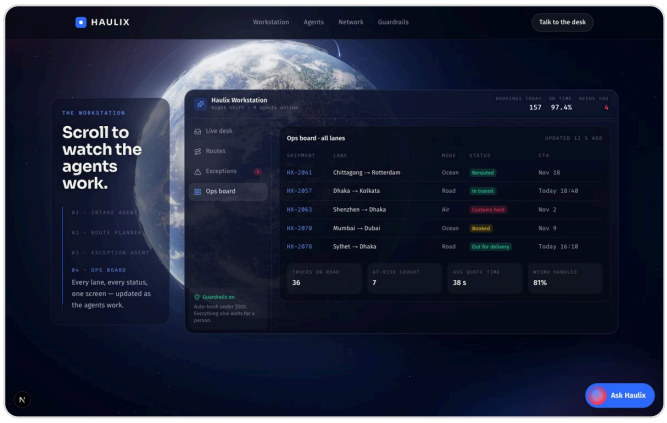
05 The workstation that follows your scroll





Stage 1 (Live desk) and stage 2 (Routes): the same panel, changed by scrolling.





Stage 3 (Exceptions: you approve or decline) and stage 4 (Ops board).

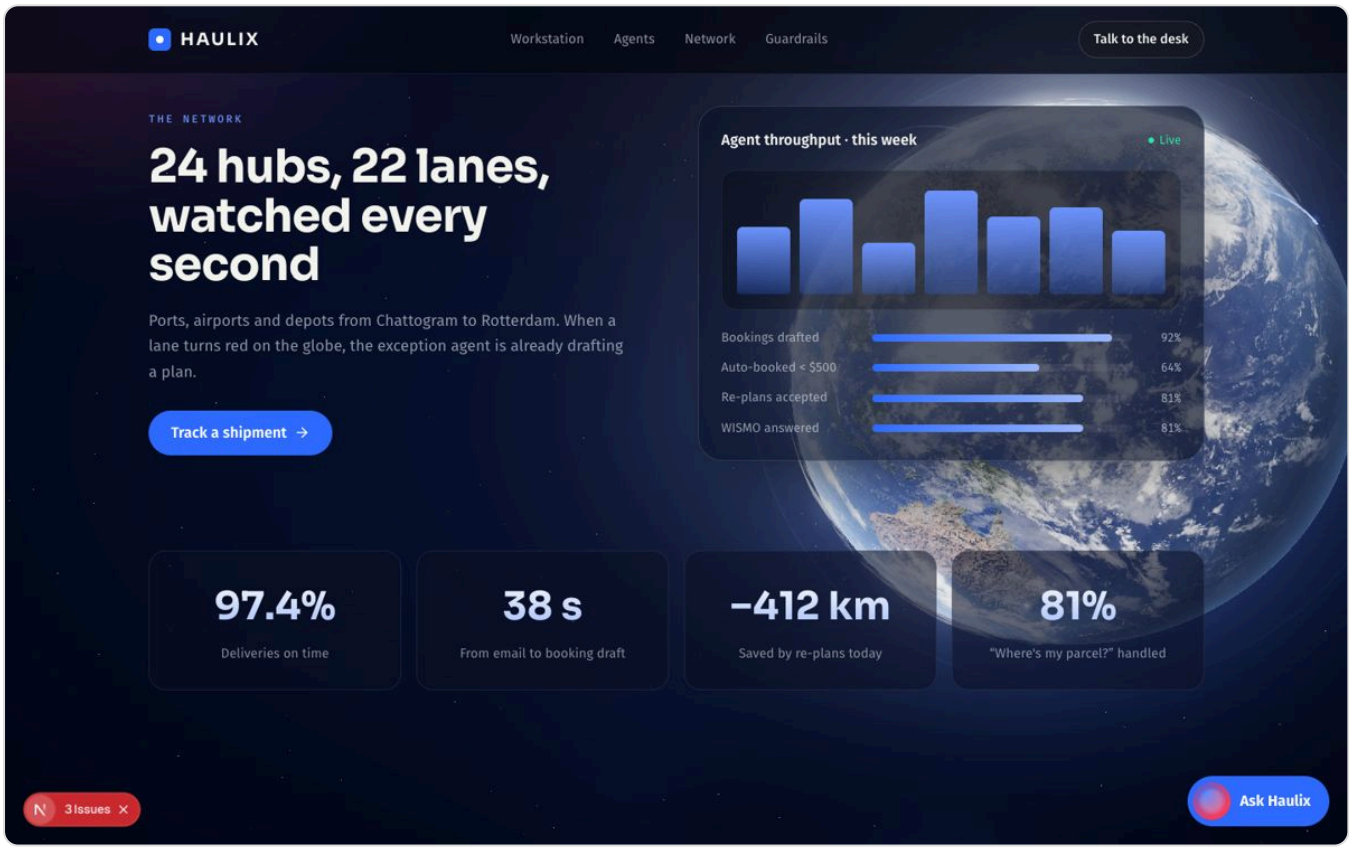
On a laptop, the workstation section is **460vh tall** (4.6 screens), but its inside is **sticky**, so the panel stays on screen. As you scroll through those 4.6 screens, a progress number goes 0 → 1, and we turn it into a stage (0, 1, 2 or 3):

```
// Sections.tsx → WorkstationSection
const { scrollYProgress } = useScroll({ target: ref, offset: ["start start", "end end"] });
useMotionValueEvent(scrollYProgress, "change", (p) => {
  const x = p * 4;            // 0 … 4
  setStage(Math.floor(x));     // which of the 4 views
  setSub(x % 1);               // how far inside that view (0 … 1)
});
<Workstation stage={stage} sub={sub} onStage={goStage} />
```

- **Live desk:** a timer adds a new message every 2.6 s (email, WhatsApp, PDF, call). 1.4 s later it turns into a booking with chips (lane, load, time). The feed has a **fixed height** so it never pushes the page.
- **Routes:** a small map drawn in SVG. When `sub` passes 0.45 the 2 pm order is added and the route is re-planned (nearest-neighbour + 2-opt — simple route maths). The line redraws with `pathLength` 0 → 1.
- **Exceptions:** each problem comes with the agent's plan, cost and new ETA, and **Approve / Decline** buttons. Your choice is remembered in state.
- **Ops board:** a table of shipments with coloured status pills.
- On phones the section is not pinned; the same component shows **tabs** instead.

06 Small effects worth copying

- **Reveal on scroll:** `Reveal` wraps any block — it starts 26 px lower and invisible, and fades up the first time it enters the screen (`whileInView`). The `d` prop delays it, so items appear one after another.
- **Counting numbers:** `Count` animates 0 → 97.4 when it first becomes visible (`useInView` + `animate`).
- **Growing bars:** each bar starts at `scaleY: 0` and grows to full height with a small delay per bar.
- **Readable text over a bright planet:** headings get a soft dark `text-shadow` (in globals.css). Cheap and invisible, but it saves legibility.
- **No scroll jumps:** `overflow-anchor: none` stops the browser from moving the page when a live widget changes size.



Network: bars grow, rows fill, numbers count up — each triggered once when it scrolls into view.

07 How to change things

I want to…	Do this
Change the headline or texts	Edit the words in `app/components/Sections.tsx` (Hero, Agents…)
Change agent names or today's numbers	`app/data/site.ts` → `AGENTS`
Change shipments in the Ops board	`app/data/site.ts` → `SHIPMENTS`
Add a new incoming order to the live desk	`Workstation.tsx` → `ORDERS` (copy one row)

I want to…	Do this
Add a new exception	`Workstation.tsx` → `EXCEPTIONS`
Change the accent colour	`app/globals.css` → `--color-radar` (and `--color-radar-soft`)
Add a port to the globe	`Planet.tsx` → `HUBS` ([latitude, longitude]); connect it in `LANES`
Make the planet spin faster	`Planet.tsx` → `spin += dt * 0.025` (bigger number = faster)

```
cd ~/Downloads/haulix-logistics
npm install          # first time only
npm run dev -- -p 3101
# open http://localhost:3101

# Publish a change: commit and push — Vercel redeploys automatically
git add -A && git commit -m "describe the change" && git push
# live: https://haulix-ai-ops-desk.vercel.app
```

08 Words you'll meet

Word	What it means here
Section / pinned section	A block of the page. *Pinned* (or *sticky*) means it stays on screen while you keep scrolling, so things inside it can animate.
Scroll progress (p)	A number from 0 to 1: how far you are through a section. Almost every animation is just “turn p into a position, size or opacity”.
Component	A reusable piece of a React page, written in a `.tsx` file (e.g. `Hero`, `Workstation`).
State	A value a component remembers (e.g. which tab is open). Changing it redraws that part of the page.
motion	The animation library: `motion.div` is a div that can animate; `useScroll` gives scroll progress; `useTransform` maps it to a value.
Lenis	Smooth scrolling: it makes the wheel feel soft and inertial instead of jumpy.
three.js / WebGL	Real 3D in the browser: a scene, a camera, lights and objects drawn by the graphics card every frame.
Shader (GLSL)	A tiny program that runs on the graphics card for every pixel (or every point) — used for special looks like night lights or glow.
Texture	An image wrapped onto a 3D object (e.g. the Earth's day map).
Glass / glassmorphism	A see-through panel: semi-transparent background + `backdrop-filter: blur()` + a thin bright border.
Tailwind classes	Short style words in `className`, e.g. `rounded-2xl` (round corners), `px-6` (side padding), `text-ink/60` (ink colour at 60%).